Program 16

```
# Input data (AND gate)
X = [
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
]

# Expected output
y = [0, 0, 0, 1]

# Create the feed-forward neural network
model = MLPClassifier(
    hidden_layer_sizes=(4,),
    activation='relu',
    solver='lbfgs',
    max_iter=2000,
    random_state=1
)

# Train the network
model.fit(X, y)

# Test the network
print("Feed Forward Neural Network")
for inputs in X:
    prediction = model.predict([inputs])
    print(inputs, "->", prediction[0])
```

Program 17

```
% Base case
sum(0, 0).

% Recursive rule
sum(N, S) :-
    N > 0,
    N1 is N - 1,
    sum(N1, S1),
    S is N + S1.
```

Program 18

```
# Name and Date of Birth Database

# Store names and DOBs in a dictionary
database = {
    "Arun": "2002-01-15",
    "Bala": "2003-06-20",
    "Chitra": "2004-09-10",
    "Devi": "2002-12-25",
    "Kumar": "2003-03-08"
}

# Display all records
print("Name and DOB Database")
print("---------------------")

for name, dob in database.items():
    print("Name:", name, "| DOB:", dob)

# Search for a person's DOB
search_name = input("\nEnter a name to search: ")

if search_name in database:
    print("Name:", search_name)
    print("Date of Birth:", database[search_name])
else:
    print("Name not found in database.")
```

Program 19

```
# Complete Student, Teacher, Subject, Name and DOB Database

# Student database: name and date of birth
students = {
    "Rahul": "2004-05-12",
    "Priya": "2005-08-20",
    "Arun": "2004-11-15",
    "Pavan": "2005-01-10"
}

# Teacher database
teachers = {
    "Ramesh": "Python",
    "Suresh": "Java",
    "Lakshmi": "DBMS",
    "Kumar": "Python"
}

# Subject database
subjects = ["Python", "Java", "DBMS"]

# Student and subject relationships
student_subjects = {
    "Rahul": "Python",
    "Priya": "Java",
    "Arun": "DBMS",
    "Pavan": "Python"
}


# Display all students with DOB
def display_students():
    print("\n--- Student Name and DOB ---")
    for name, dob in students.items():
        print("Name:", name, "| DOB:", dob)


# Display all teachers and their subjects
def display_teachers():
    print("\n--- Teacher and Subject ---")
    for teacher, subject in teachers.items():
        print("Teacher:", teacher, "| Subject:", subject)


# Display all subjects
def display_subjects():
    print("\n--- Available Subjects ---")
    for subject in subjects:
        print(subject)


# Search student details
def search_student():
    name = input("Enter student name: ").strip().title()
    if name in students:
        print("Name:", name)
        print("DOB:", students[name])
        print("Subject:", student_subjects[name])
    else:
        print("Student not found.")


# Find teacher for a subject
```

```
def search_teacher():
    subject = input("Enter subject: ").strip().lower()
    found = False

    for teacher, taught_subject in teachers.items():
        if taught_subject.lower() == subject:
            print("Teacher:", teacher)
            found = True

    if not found:
        print("No teacher found for this subject.")


# Main menu
while True:
    print("\n===== DATABASE MENU =====")
    print("1. Display Students and DOB")
    print("2. Display Teachers and Subjects")
    print("3. Display All Subjects")
    print("4. Search Student Details")
    print("5. Search Teacher by Subject")
    print("6. Exit")

    choice = input("Enter your choice: ").strip()

    if choice == "1":
        display_students()
    elif choice == "2":
        display_teachers()
    elif choice == "3":
        display_subjects()
    elif choice == "4":
        search_student()
    elif choice == "5":
        search_teacher()
    elif choice == "6":
        print("Program ended.")
        break
    else:
        print("Invalid choice. Please try again.")
```

Program 20

```
# Planets Database in Python

# Planet database: name, position, and type
planets = {
    "Mercury": {"position": 1, "type": "Terrestrial"},
    "Venus": {"position": 2, "type": "Terrestrial"},
    "Earth": {"position": 3, "type": "Terrestrial"},
    "Mars": {"position": 4, "type": "Terrestrial"},
    "Jupiter": {"position": 5, "type": "Gas Giant"},
    "Saturn": {"position": 6, "type": "Gas Giant"},
    "Uranus": {"position": 7, "type": "Ice Giant"},
    "Neptune": {"position": 8, "type": "Ice Giant"}
}

# Display all planets
def display_planets():
    print("\n--- Planets Database ---")
    for name, details in planets.items():
        print("Planet:", name,
              "| Position:", details["position"],
              "| Type:", details["type"])

# Search for a planet
def search_planet():
    name = input("Enter planet name: ").strip().title()

    if name in planets:
        print("\nPlanet:", name)
        print("Position from Sun:", planets[name]["position"])
        print("Type:", planets[name]["type"])
    else:
        print("Planet not found.")

# Display planets by type
def search_by_type():
    planet_type = input("Enter type (Terrestrial, Gas Giant, Ice Giant): ").strip().lower()

    found = False
    for name, details in planets.items():
        if details["type"].lower() == planet_type:
            print(name)
            found = True

    if not found:
        print("No planets found for this type.")

# Main menu
while True:
    print("\n===== PLANETS DATABASE =====")
    print("1. Display all planets")
    print("2. Search for a planet")
    print("3. Search planets by type")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        display_planets()
    elif choice == "2":
        search_planet()
    elif choice == "3":
        search_by_type()
    elif choice == "4":
```

```
    print("Thank you!")
    break
else:
    print("Invalid choice. Try again.")
```

Program 21

```
def tower_of_hanoi(n, source, auxiliary, destination):
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return

    tower_of_hanoi(n - 1, source, destination, auxiliary)

    print(f"Move disk {n} from {source} to {destination}")

    tower_of_hanoi(n - 1, auxiliary, source, destination)


# Main program
n = int(input("Enter the number of disks: "))

tower_of_hanoi(n, 'A', 'B', 'C')
```

Program 22

```
# Facts
birds = ["parrot", "pigeon", "sparrow", "ostrich"]

# Rule
def can_fly(bird):
    if bird in birds and bird != "ostrich":
        return True
    else:
        return False

# Input
bird = input("Enter a bird name: ").lower()

# Output
if can_fly(bird):
    print(bird, "can fly.")
elif bird in birds:
    print(bird, "cannot fly.")
else:
    print(bird, "is not in the bird database.")
```

Program 23

```
# Facts
parents = {
    "john": ["mary", "david"],
    "mary": ["alice", "bob"],
    "david": ["charlie"]
}

# Rule: Find children of a person
def children(person):
    return parents.get(person, [])

# Rule: Find the parent of a person
def find_parent(child):
    for parent, kids in parents.items():
        if child in kids:
            return parent
    return None

# Main program
name = input("Enter a person's name: ").lower()

parent = find_parent(name)

if parent:
    print(name, "is the child of", parent)
else:
    print("Parent not found for", name)

kids = children(name)

if kids:
    print(name, "is the parent of:", ", ".join(kids))
else:
    print(name, "has no children in the family tree.")
```

Program 24

```
# Diet Suggestion System Based on Disease

def diet_suggestion(disease):
    if disease == "diabetes":
        print("Suggested diet: Whole grains, vegetables, pulses, and low-sugar fruits.")

    elif disease == "hypertension":
        print("Suggested diet: Fruits, vegetables, and low-sodium foods.")

    elif disease == "anemia":
        print("Suggested diet: Iron-rich foods such as spinach, beans, and lentils.")

    elif disease == "obesity":
        print("Suggested diet: Vegetables, fruits, whole grains, and balanced portions.")

    elif disease == "heart disease":
        print("Suggested diet: Vegetables, whole grains, and foods low in saturated fat.")

    else:
        print("No diet suggestion available for this condition.")

# Main program
print("Diet Suggestion System")
print("Available conditions: diabetes, hypertension, anemia, obesity, heart disease")

disease = input("Enter the disease: ").lower()

diet_suggestion(disease)
```

Program 25

```
# Monkey and Banana Problem

monkey = "at_door"
box = "at_window"
banana = "hanging"
has_banana = False

print("Initial State:")
print("Monkey is at the door.")
print("Box is at the window.")
print("Banana is hanging from the ceiling.")

# Step 1: Monkey moves to the box
monkey = "at_window"
print("\nMonkey moves to the window.")

# Step 2: Monkey pushes the box under the banana
box = "under_banana"
monkey = "under_banana"
print("Monkey pushes the box under the banana.")

# Step 3: Monkey climbs onto the box
monkey = "on_box"
print("Monkey climbs onto the box.")

# Step 4: Monkey grabs the banana
if monkey == "on_box" and box == "under_banana":
    has_banana = True
    print("Monkey grabs the banana!")

# Final State
if has_banana:
    print("\nGoal Achieved: Monkey has the banana.")
else:
    print("\nGoal not achieved.")
```